

Advanced C, part 3 of 3



Copyright Brian Brown, 1986-1999. All rights reserved.

[Comprehensive listing of interrupts and hardware details.](#)

CONTENTS OF PART THREE

- [Floppy Disk Calls](#)
- [Sound Generation](#)
- [LongJump and ctrl-break handling](#)
- [Interrupt handling](#)
- [Embedded Code](#)
- [Make utility](#)
- [DOS Interrupts](#)
- [Interfacing to mouse.sys](#)
- [Interfacing to a joystick](#)
- [Video Library Routines](#)

FLOPPY DISK SERVICES

INT 13H This interrupt provides for the management of the floppy disk drive and controller unit. The AH register value, on entry, determines the desired function.

```

AH = 0 Reset Floppy Disk System No return value
AH = 1 Get Status Returns AH as a status byte,
      Bit 7 = time-out
      6 = seek failure
      5 = controller error
      4 = CRC error
      3 = DMA overrun
      2 = sector not found
      1 = write-protected disk
      0 = illegal command
AH = 2 Read Sector(s)
      AL = number of sectors to transfer (1-9)
      ES:BX = segment:offset of disk I/O buffer
      CH = track number (0-39)
      CL = sector number (1-9)
      DH = head number (0-1)
      DL = drive number (0-3)
      Returns on success cflag = clear
           AH = 0
           AL = number of disk sectors transferred
           on failure cflag = set
           AH = status byte
AH = 3 Write Sector(s) same as for read sector(s)
AH = 4 Verify Sector(s)
      AL = number of sectors to transfer (1-9)
      CH = track number (0-39)
      CL = sector number (1-9)
      DH = head number (0-1)

```

```

    DL = drive number (0-3)
    Returns on success cflag = clear
        AH = 0
        AL = number of disk sectors transferred
    on failure cflag = set
        AH = status byte
AH = 5 Format Track
    ES:BX = segment:offset of address field list
    No return value

```

SOUND GENERATION

Port 43h provides access to the registers of port 42h. First store the magic value 0xb6 to port 43h, then load two 8 bit values into port 42h, which specify the frequency to generate. Once this is done, turning on bits 1 and 0 of port 61h will enable the circuitry and produce the tone. Summarising the steps, they are

- 1: Output 0xb6 to port 43h
- 2: Send each of the 8 bit values to port 42h
- 3: Enable bits 0 and 1 of port 61h

Step 1 is achieved by the C statement `outportb( 0x43, 0xb6 );`

Step 2 is achieved by converting the frequency into two eight bit values, then outputting them to port 42h. The Most Significant Byte is sent last.

```

Frequency required to generate = 512 hertz
16 bit value = 120000h / frequency

```

so, this is achieved by the C statements

```

outportb(0x42, 0)
outportb(0x42, 6)

```

Step 3 is achieved by the C statements

```

byte = inportb(0x61);
byte |= 3;
outportb(0x61, byte);

```

Connecting this together into a function, it becomes,

```

void tone_512() {
    char byte;
    outportb(0x43, 0xb6);
    outportb(0x42, 0);
    outportb(0x42, 6);
    byte = inportb(0x61);
    byte |= 3;
    outportb(0x61, byte);
}

```

```
}
```

There follows two routines to generate sound using the timer chip. The first, beep(), sounds a note of 1000hz for a short duration. The second, note() allows you to specify the frequency and duration of the desired note.

```
#include <dos.h>
void beep() {
    int delay;
    _AL = 0xb6;
    outportb(0x43,_AL); /* write to timer mode register */
    _AX = 0x533; /* divisor for 1000hz */
    outportb(0x42,_AL); /* write LSB */
    _AL = _AH;
    outportb(0x42,_AL); /* write MSB */
    _AL = inportb(0x61); /* get current port setting */
    _AH = _AL; /* save it in _AH */
    _AL |= 3; /* turn speaker on */
    outportb(0x61,_AL);
    for(delay = 0; delay < 20000; delay++)
        ;
    _AL = _AH;
    outportb(0x61,_AL); /* restore original settings */
}

int note( frequency, duration )
unsigned int frequency, duration;
{
    unsigned long delay;
    if( frequency > 5000 ) return 1;
    _AL = 0xb6;
    outportb(0x43,_AL); /* write to timer mode register */
    _AX = 0x12000L / frequency; /* calculate divisor required */
    outportb(0x42,_AL); /* write LSB */
    _AL = _AH;
    outportb(0x42,_AL); /* write MSB */
    _AL = inportb(0x61); /* get current port setting */
    _AL |= 3; /* turn speaker on */
    outportb(0x61,_AL);
    for(delay = 0L; delay < (long) duration * 45; delay++)
        ;
    _AL = inportb(0x61); /* turn off sound */
    _AL &= 0xfc; /* set bits 1 and 0 off */
    outportb(0x61,_AL); /* restore original settings */
    return 0;
}

main() {
    unsigned int f;
    for(f = 100; f < 250; f += 10 ) {
        note( f, (unsigned int) 1000 ); /* 1000 = 1 second */
    }
}
```

LONGJUMP/SETJUMP/CTRL-BRK

The purpose of this section is to illustrate the techniques involved in taking over the control-break and control-c routines. We will show you how to enable and disable control-c checking. As well,

features of the longjmp/setjmp routines will be demonstrated.

Control/Break

The routine which controls the detection of control-break resides in the ROM BIOS chip (int 16h), thus cannot be disabled. The keyboard interrupt routine, upon detecting a Ctrl-Break, generates an interrupt into DOS (type 1bh). It is thus possible to re-direct this DOS interrupt to your own routine.

Control/C

Ctrl-C is detected by DOS. This may be disabled or enabled using the setcbrk() in TurboC. The function ctrlbrk() allows redirection of the Ctrl-C interrupt (int 23h) to a particular function.

Lets build up some routines similar to those found in the TurboC library.

```
#include <dos.h>

int setcbrk( value ) /* set control c checking, 0 = off, 1 = on */
int value;
{
    union REGS regs;
    regs.h.ah = 0x33;
    regs.h.al = 01;
    regs.h.dl = value;
    intdos( &regs, &regs );
}

int getcbrk() /* get control C status, 0 =off, 1 = on */
{
    union REGS regs;
    regs.h.ah = 0x33;
    regs.h.al = 00;
    intdos( &regs, &regs );
    return( regs.h.dl & 1 );
}
```

The following program illustrates the use of re-directing the ctrl-c interrupt 0x23 to a user supplied routine (TurboC only).

```
#include <dos.h>

int ctc() /* exits to DOS if return value is 0 */
{
    static int times=0;
    printf("ctc is activated %d.\n", times);
    ++times;
    if(times >= 5) return(0);
    else return(1);
}

main() {
    int value;
    value = getcbrk();
    printf("Control-C checking is ");
    if( value )
        printf("on");
    else
        printf("off");
    printf("\nRedirecting ctrl-c to user routine ctc.\n");
}
```

```

        ctrlbrk(ctc);
        for( ; ; )
            printf("Press ctrl-c to exit (5)\n");
    }

```

LONGJMP and SETJMP

These routines allow processing to restart from a designated part of the program. Examples of this might be a menu driven system, which has many layers. A user, accessing a low level menu, by pressing ctrl-c, could immediately be placed into the highest level menu as though the package had just been restarted! Lets show how this is done by way of an example.

```

#include <setjmp.h> /* for setjmp and longjmp */
#include <stdio.h>

jmp_buf jumper; /* for an image of the stack frame entries */

void getkey() {
    printf("Press any key to continue.\n"); getchar();
}

int ctrlbreak() {
    printf("\n. Returning to main menu now.\n"); getkey();
    longjmp( jumper, 1 );
}

main() {
    ctrlbrk( ctrlbreak ); /* set up control-c handler routine */
    setjmp( jumper ); /* remember this as the entry point */
    for( ; ; ) { /* will return here when user press's ctrl-brk */
        printf("Top menu.\nPress any key"); if( getchar()=='E') e
        for( ; ; ) {
            printf("Menu 2.\nPress any key"); getchar();
            for( ; ; ) {
                printf("Menu 3.\nPress any key"); getchar
            }
        }
    }
}

```

INTERRUPT ROUTINES

This section concentrates upon writing interrupt routines to replace those resident by either DOS or the ROM BIOS. By way of an illustration, we will show you how to take over the shift/prtscrn interrupt, which dumps the screen to the printer. This may be useful on a machine which does not have a printer. TurboC will be used to demonstrate this technique. Once this has been done, we will also show you how to modify it so that it stays resident in memory, rather than just lasting whilst the program lasts.

```

#include <dos.h>

void interrupt (*old_int5)();

void interrupt my_int5( unsigned bp, unsigned di, unsigned si,

```

```

unsigned ds, unsigned es, unsigned dx,
unsigned cx, unsigned bx, unsigned ax )
{
    /* normally, place nothing here, just a dummy routine */
    _AH = 0x0a;
    _AL = '5';
    _CX = 80;
    geninterrupt( 0x10 );
}

int ctrlbreak() {
    printf("\n. Returning to DOS now.\n");
    setvect( 5, old_int5 ); /* restore original vector */
    return( 0 );
}

main() {
    ctrlbrk( ctrlbreak ); /* set up control-c handler routine */
    old_int5 = getvect( 5 );
    printf("Resetting int_5 now.\n");
    setvect( 5, my_int5);
    for( ; ; )
        printf("Press ctrl-c to exit, shift-prtscrn to test.\n");
}

```

Be very careful about the use of DOS routines inside your interrupt routines. Calls to printf(), scanf() etc will probably result in a system crash. Now, lets present the above program as a terminate and stay resident program.

```

/* compiled in TurboC V1.0, using Large Memory Model */
#include <dos.h>

extern void far *_heapbase;

void interrupt my_int5( unsigned bp, unsigned di, unsigned si,
unsigned ds, unsigned es, unsigned dx,
unsigned cx, unsigned bx, unsigned ax )
{
}

main() {
    setvect( 5, my_int5);
    keep( 0, FP_SEG(_heapbase) - _psp );
}

```

Programs which Terminate and Stay Resident (TSR) are not simple. How-ever, there have been some good articles written recently concerning this.

```

Writing TSR's in TurboC : Al Stevens
(Computer Language, Feb 1988)
Converting TC programs to a TSR : M Young
(DOS Programmers Journal 1988, v6.2)

```

Now lets develop a program which is slightly more sophisticated. This program displays the time in the left top corner of the video screen.

```

#include <stdio.h> /* timer.c, (c) B Brown, 1988 */
#include <dos.h> /* TurboC V1.0, Large memory model */

```

```

#include <string.h>

extern void far *_heapbase;
static unsigned int TCSS;
static unsigned int TCSP;
static unsigned int TCDS;
static unsigned int OLDSS;
static unsigned int OLDSP;
static unsigned int OLDDS;
static int far *tbase = (int far *) 0x0000046c1;
static void interrupt (*oldtimer)();
static char buffer[20];
static int loop, xpos, ypos, vpage = 0;

static struct t {
    unsigned int sec, min, hor;
} tme;

void interrupt mytime( unsigned bp, unsigned di, unsigned si,
unsigned ds, unsigned es, unsigned dx,
unsigned cx, unsigned bx, unsigned ax )
{
    /* save old values of registers, get programs stack */
    disable();
    OLDSS = _SS;
    OLDSP = _SP;
    OLDDS = _DS;
    _DS = TCDS;
    _SS = TCSS;
    _SP = TCSP;
    /* get timer values */
    tme.hor = *(tbase + 1);
    tme.min = (*tbase / 18) / 60;
    tme.sec = (*tbase / 18) - (tme.min * 60);
    /* convert values to a character string */
    buffer[0] = (tme.hor / 10) + '0';
    buffer[1] = (tme.hor % 10) + '0';
    buffer[2] = ':';
    buffer[3] = ((tme.min / 10) | 0x30);
    buffer[4] = ((tme.min % 10) | 0x30) - 1;
    buffer[5] = ':';
    buffer[6] = ((tme.sec / 10) | 0x30);
    buffer[7] = (tme.sec % 10) + '0';
    buffer[8] = '\\0';
    enable();
    /* save current cursor position */
    _AH = 3; _BH = vpage; geninterrupt(0x10); xpos = _DL; ypos = _DH;
    /* set cursor to row 0, column 0 */
    _AH = 2; _BH = vpage; _DX = 0; geninterrupt( 0x10 );
    /* print time on screen */
    for( loop = 0; loop < 8; loop++ ) {
        _AH = 0x0a; _AL = buffer[loop]; _BH = vpage;
        _CX = 1; geninterrupt( 0x10 );
        _AH = 2; _BH = vpage; _DX = loop + 1; geninterrupt(0x10);
    }
    /* restore original cursor position */
    _AH = 2; _BH = vpage; _DH = ypos, _DL = xpos; geninterrupt( 0x10
    /* chain to old timer interrupt */
    (*oldtimer)();
    /* restore register values, calling stack etc */
    _SS = OLDSS;
    _SP = OLDSP;
    _DS = OLDDS;
}

```

```

main() {
    disable();
    oldtimer = getvect( 0x1c ); /* get original vector */
    disable();
    TCSS = _SS; /* save segment values etc of programs stack */
    TCSP = _SP;
    TCDS = _DS;
    setvect( 0x1c, mytime ); /* hook into timer routine vector */
    enable();
    keep( 0, FP_SEG(_heapbase) - _psp ); /* tsr */
}

```

PRODUCING EMBEDDED CODE

Using the PC as a stand-alone system such as a data-logger, terminal etc, poses several problems. Generally, the software will be written to reside at a specific place in memory, usually in an EPROM. When the PC is turned on, this software is activated. This means that DOS is probably not present. If the software is written with any calls to DOS (examples being printf(), scanf() etc), then it will certainly crash.

Should the ROM BIOS chip be left on board, then calls to it via int xxh will probably work okay. This depends very much upon where the software is located in memory. As I see it, there are several options open. On a 640k RAM machine, RAM goes from 00000 to 9ffff, with the graphic cards going from a0000 to bffff. This leaves user ROM space from c0000 up to f4000.

Depending upon the ROM BIOS chip, some routines are not executed if you place code between c0000 to c7fff. These routines are probably initialisation of the keyboard queue, video display and disk drive controller (which may be important if you intend to use int 16h, int 10h and int 13h). Manufacturers of EGA cards, hard disk drives, lan cards etc usually place their code between c8000 to f4000.

On power up, as the ROM BIOS is being executed, it first checks for ROM chips between c0000 to c7fff, at every 2k boundary. If it finds one, it will leap to the ROMS entry address and execute the code there. Upon return (or if it doesn't find a ROM chip), it initialises the keyboard queue and video display, then checks for ROM between c8000 to f4000. If it finds a ROM here, it again calls it to execute the code.

If no ROM chips are found, the computer will attempt an int19h (Bootstrap Loader routine). If this is unsuccessful, an int18h instruction will be generated (a call to F6000, ie, BASIC ROM). If there are no BASIC ROM chips on board, it's likely that the system will perform a reset.

BASIC ROM resides from f4000 up, the entry point is f6000. The BIOS ROM resides from fe000 to fffff (normally an 8k EPROM, type 2764).

The format of User ROM chips residing between c0000 to f4000

If you decide to create a program which resides in this address space, then download it into an EPROM for placement on a board, its important to adhere to special provisions concerning the format of the initial 5 bytes of code. A ROM chip must be identified with the bytes 55 and AA in the first 2 locations, followed by a byte which represents the length of the program divided by 512, then followed by an instruction or jump to the entry routine (initialisation code, which sets up the segment register values, stack space etc).

The process of generating ROMMABLE code.

Rommable code is created by either specifying the absolute segment addresses using assembler (segment at 0c800h), or using a LOCATOR program which assigns addresses to the various segment groups once the program has been linked. This creates an absolute image file which can be downloaded into an EPROM programmer unit, which then programs the actual EPROM chip.

Other Considerations

How-ever, there are many traps involved in writing embedded code. Lets look at some of these to start with.

Library Routines

The library routines supplied with most compilers use DOS to perform video, keyboard and diskette functions. Your own versions will need to be created instead. Access to the source code for library functions will be helpful. If the ROM BIOS chip is left in place, it should be easy to write routines which substitute for the library.

Segment register values

With plenty of interrupts running around, it is important that you initialise the segment registers (DS,ES and SS,SP) when the jump to your ROM code takes place. Failure to do so can result in stack overflow, and the inability to access any data. Create your own stack somewhere safe in memory first.

Copy data to RAM

Copy your initialised data to RAM, and don't forget to adjust the DS or ES register to point to the segment address! Zero out any RAM block used for uninitialised static variables (ie, having the value 0).

Plug into the interrupt vectors

You may safely take over most interrupt routines, including int 10h etc. You will need to write your own routines to do this, don't rely upon the library functions which come with your compiler. The final section of this booklet demonstrates this. Ensure that interrupt routines which are called are type FAR, and save all the registers. Interrupt routines should also set up segment register values for DS/ES, if they need to access data some-where else.

Here are a couple of ROMMable routines

```
void set_vect( int_number, int_code )
unsigned int int_number;
long int *int_code;
{
    unsigned int seg, off;
    int far *int_vector = (int far *) 0x00000000;
    seg = FP_SEG( int_code );
    off = FP_OFF( int_code );
    number &= 0xff;
    number <= 2;
    *int_vector[number] = off;
    *int_vector[number+2] = seg;
}

print_str proc near ; ROM Version of int21h, ah = 09
    push si ; use si for indexed addressing
    push ax ; save character
    mov ax,dx ; establish addressability
    mov si,ax ; dx has offset of string from DS
    pstr1: mov al,[si] ; get character
    cmp al,24h ; is it end of string
```

```

        je pstr2 ; yes then exit
        call pc_out ; no, print it then
        inc si ; point to next character
        jmp pstr1 ; repeat
pstr2: pop ax
        pop si
        ret
print_str endp

pc_out proc near ; print character on video screen
        mov ah,0eh ; write tty call
        push bx
        push cx
        mov bh,0 ; assume page zero
        mov cx,1 ; one character to write
        int 10h
        pop cx
        pop bx
        ret
pc_out endp

```

Keyboard Removal

Some BIOS routines check for keyboard existence prior to checking for user EPROM. If intending to run with the keyboard removed, and the BIOS chips present, either the keyboard must be present during a system reset, or the BIOS will need to be modified.

Running without the BIOS chips

You will need to initialise

- the interrupt vectors (maybe to an iret instruction)
- timer 0 to perform refreshing of dynamic ram
- the DMA and Priority Interrupt Controller (PIC) devices
- RAM to zero so that the parity generator doesn't get confused
- the segment registers SS, DS and ES to their respective areas
- any additional cards or devices used, video, keyboard, disk, A/D

You will need to test

- CPU
- for top of memory
- RAM and ROM
- interrupts working (PIC)
- timer channel operation

MAKE

This is a facility which offers project management of multiple source and object files. A special file (makefile), contains the files which the runtime code is dependant upon. When make is invoked, it checks the date of each file, and decides which files need re-compiling or re-linking.

Create a makefile Use an editor to create makefile, eg

```
$vi makefile
```

In makefile, place the following, ensuring that tab stops are placed between myprog.exe and myprog.obj, and between the left margin and tlink.

```
myprog.exe: myprog.obj f1.obj f2.obj f3.obj
tlink c01 myprog f1 f2 f3, myprog, myprog, c1
myprog.obj: myprog.c f1.c f2.c f3.c
tcc -c -ml -f- myprog.c
f1.obj: f1.c
tcc -c -ml -f- f1.c
f2.obj: f2.c
tcc -c -ml -f- f2.c
f3.obj: f3.c
tcc -c -ml -f- f3.c
```

Now, create the following modules f1.c f2.c f3.c and myprog.c

```
void print_mess1() /* Module f1.c */
{
    printf("This is module f1\n");
}

void print_mess2() /* Module f2.c */
{
    printf("This is module f2\n");
}

void print_mess3() /* Module f3.c */
{
    printf("This is module f3\n");
}

extern void print_mess1(), print_mess2(), print_mess3();
main() /* Module myprog.c */
{
    print_mess1();
    print_mess2();
    print_mess3();
    printf("and this is main\n");
}
```

Compile each of the above modules, using the tcc stand-alone compiler.

```
$tcc -c -ml -f- myprog.c
$tcc -c -ml -f- f1.c
$tcc -c -ml -f- f2.c
$tcc -c -ml -f- f3.c
```

Now you are in a position to try out the make function.

```
$make
```

This runs the make utility, which will receive as its input the file contents of makefile, and generate the required runfile myprog.exe

```
$myprog
This is module f1
This is module f2
```

```
This is module f3
and this is main
$
```

If changes are made to any of the source files from this point on, you only need to re-run make. This helps to automate the process of program maintenance. It is possible to specify a command file other than makefile, which contains inter-dependencies, eg,

```
$make myprog
```

will perform a make on the inter-dependant commands specified in the file myprog.

DOS INTERRUPT ROUTINES

To maintain compatibility across different hardware machines and interfaces, calls to the DOS are preferable to the low level access provided by the ROM BIOS routines. Two routines allow access to the DOS interrupt interface. They are `intdos()` and `intdosx()`. Both functions generate a DOS interrupt type 0x21.

```
intdos( union REGS *regs, union REGS *regs)
intdosx( union REGS *regs, union REGS *regs, struct SREGS *segregs )
```

The function `intdosx()` also copies the values for the `segregs.x.ds` and `segregs.x.es` into the DS and ES registers. Both functions copy the register values returned from the DOS call into the associated structure, as well as the status of the carry flag. If the carry flag is set, this indicates an error. The following functions illustrate calls using the DOS interrupt interface.

```
#include <dos.h>

union REGS regs;

char rs232_read() {
    regs.h.ah = 3; intdos( &regs, &regs ); return( regs.h.al );
}

void rs232_write( byte )
char byte;
{
    regs.h.ah = 4; intdos( &regs, &regs );
}
```

When a C program is run, DOS opens five pre-defined streams for the program to access. The streams are used with the C functions `open()`, `read()`, `write()`, and `close()`. The five pre-defined streams are,

```
0=CON stdin 1=CON stdout
2=CON stderr 3=AUX stdaux COM1
4=PRN stdprn LPT1
```

A program may access these opened streams, however, direct reads and writes can fail due to DOS re-direction. It is best to re-open device first, before performing any operations. This will prevent any

re-direction.

The following code portion shows how to write to the prn device from within a C program.

```
#include <fcntl.h>
#include <string.h>
#include <io.h>

main() {
    char *message = "This is a message for the prn device.";
    int prnhandle;
    if( (prnhandle = open( "PRN", O_WRONLY, O_BINARY) ) == -1 ) {
        printf("Couldn't open prn device.\n");
        exit( 1 );
    }
    printf("Printer is on-line. Now printing the message.\n");
    if( (write( prnhandle, message, strlen(message) )) == -1 ) {
        printf("write to prn device failed.\n");
        exit(2);
    }
    printf("Message has been printed. Lets close and exit to DOS.\n");
    close( prnhandle );
}
```

INTERFACING TO MOUSE.SYS

The mouse driver locates itself in memory at boot time. It takes over both int 33h and int 10h. The driver is identified by an eight character sequence, in the case of the microsoft mouse, it is the sequence MS\$MOUSE. Before issuing any calls to the mouse driver, you should first establish its presence. There are two methods of accomplishing this. First, you can test to see if the driver is installed by checking for the device name, or use a mouse call to int 33h. The routine which follows returns 0 if the mouse driver does not exist, -1 if it is present.

```
#include <dos.h>

int mouse_exist2( void ) {
    _AX = 0;
    geninterrupt( 0x33 );
    return _AX;
    /* _BX will also contain the number of buttons */
}
```

The mouse_exist2() call also initialises the mouse system to the default parameters, if it is present.

Mouse Function Calls INT 33h

```
AX = 0 Mouse Installed Flag and RESET
Returns AX as a status byte, 0 = not present, -1 = present (and R
The default parameters for a RESET are,
cursor position = screen centre
internal cursor flag = -1 (not displayed)
graphics cursor = arrow (-1, -1)
text cursor = inverting box
interrupt mask call = all 0 (no interrupts)
light pen emulation = enabled
```

```

        mouse/pixel ratio (H)= 8 to 8
        mouse/pixel ratio (V)= 16 to 8
        min/max cursor pos H = Depends upon card/mode
        min/max cursor pos V = Depends upon card/mode
AX = 1 Show Cursor
        Increments the internal cursor flag, and if zero, displays the cu
AX = 2 Hide Cursor
        Decrements the internal cursor flag, and removes the cursor from
AX = 3 Get Mouse Position and Button Status
        Returns the state of the left and right mouse buttons, as well as
        BX bit 0 = left button (1=pressed, 0=released)
        BX bit 1 = right button
        CX = cursor position, horizontal
        DX = cursor position, vertical
AX = 4 Set Mouse Cursor Position
        Upon entry, CX = new horizontal position
        DX = new vertical position
AX = 5 Get Mouse Button Press Information
        Upon entry, BX = which button to check for, (0=left,1=right)
        Returns the following information.
            AX = button status, bit 0 = left button
            bit 1 = right button (1=pressed, 0=released)
            BX = count of button presses (0 to 32767, reset to 0 afte
            CX = cursor position, horizontal, at last press
            DX = cursor position, vertical, at last press
AX = 6 Get Button Release Information
        Upon entry, BX = which button to check for, (0=left,1=right)
        Returns the following information.
            AX = button status, bit 0 = left button
            bit 1 = right button (1=pressed, 0=released)
            BX = count of button releases (0 to 32767, reset to 0 aft
            CX = cursor position, horizontal, at last release
            DX = cursor position, vertical, at last release
AX = 7 Set Minimum and Maximum Horizontal Position
        Upon entry, CX = minimum position
        DX = maximum position
AX = 8 Set Minimum and Maximum Vertical Position
        Upon entry, CX = minimum position
        DX = maximum position
AX = 9 Set Graphics Cursor Block
        Upon entry, BX = cursor hot spot (horizontal)
        CX = cursor hot spot (vertical)
        DX = pointer to screen and cursor masks
AX = 10 Set Text Cursor
        Upon entry, BX = cursor select (0=software, 1=hardware)
        CX = screen mask or scan line start
        DX = cursor mask or scan line end
AX = 11 Read Mouse Motion Counters
        Return values,
            CX = horizontal count
            DX = vertical count
AX = 12 Set User-Defined Subroutine Input Mask
        Upon entry, CX = call mask
        DX = address offset to subroutine
        ES = address segment to subroutine
        Each bit of the call mask corresponds to
            0 = Cursor position change
            1 = Left button pressed
            2 = Left button released
            3 = Right button pressed
            4 = Right button released
            5-15 = Not used
        To enable an interrupt, set the corresponding bit to a 1.
        When the event occurs, the mouse driver will call your subroutine

```

```

AX = 13 Light Pen Emulation Mode ON
AX = 14 Light Pen Emulation Mode OFF
AX = 15 Set Mickey/Pixel Ratio
    Upon entry, CX = horizontal ratio
    DX = vertical ratio
    The ratios specify the number of mickeys per 8 pixels. The values
    must be within the range 1 to 32767. The default horizontal ratio
    is 8:8, whilst the default ratio for the vertical is 16:8
AX = 16 Conditional OFF
    Upon entry, CX = upper x screen co-ordinate
    DX = upper y screen co-ordinate
    SI = lower x screen co-ordinate
    DI = lower y screen co-ordinate
    This function defines a region on the screen for updating. If the
    moves to the defined region, it will be hidden while the region i
    After calling this function, you must call function 1 again to sh
    cursor.
AX = 19 Set Double Speed Threshold
    Upon entry, DX = threshold speed in mickeys/second
    This function can be used to double the cursors motion on the scr
    The default value is 64 mickeys/second.

```

Mouse Demonstration Program

```

/*mousedem.c, an illustration of how to interface to mouse.sys */
/* by B Brown, 1988 */
#include <dos.h>

static unsigned int arrow[2][16] = {
    { 0xffff0, 0xfffe0, 0xfffc0, 0xff81, 0xff03, 0x607, 0xf, 0x1f, 0xc03
      0xf07f, 0xfffff, 0xfffff, 0xfffff, 0xfffff, 0xfffff, 0xfffff } ,
    { 0, 6, 0x0c, 0x18, 0x30, 0x60, 0x70c0, 0x1d80, 0x700, 0, 0, 0, 0
      0, 0, 0 }
};

void set640_200() {
    _AH = 0; _AL = 6; geninterrupt( 0x10 );
}

int mouse_exist() {
    _AX = 0; geninterrupt( 0x33 ); return _AX;
    /* _BX will also contain the number of buttons */
}

void show_cursor() {
    _AX = 1; geninterrupt( 0x33 );
}

void shape_cursor( buffer, hchs, vchs )
unsigned int *buffer;
unsigned int hchs, vchs;
{
    _AX = 9; _BX = hchs; _CX = vchs;
    _DX = FP_OFF( buffer ); _ES = FP_SEG( buffer );
    geninterrupt( 0x33 );
}

main() {
    if( mouse_exist() == 0 ) {
        printf("Mouse driver is not loaded. Returning to DOS.\n");
        exit(1);
    }
    set640_200();
    shape_cursor( arrow, 0, 0 );
}

```

```

        show_cursor();
        while( 1 )
            ;
    }

```

APPENDIX A

```

VIDEO LIBRARY ROUTINES, C Source code
/* C_UTILITIES FOR IBM-PC, MICROSOFT C COMPILER V3.0 */
/* */
/* Written by B. Brown */
/* Central Institute of Technology, */
/* Private Bag, Trentham */
/* Wellington, New Zealand, */
/* */
/* The routines are listed as follows, */
/* */
/* getmode() returns screen mode setting into */
/* 'screenmode', the number of columns */
/* into 'columns', and screen page number */
/* into 'activepage' */
/* */
/* setmode() sets the video mode based on the value */
/* of screenmode, where */
/* 0 =40x25BW 1 =40x25CO 2 =80x25BW */
/* 3 =80x25CO 4 =320x200CO 5 =320x200BW */
/* 6 =640x200BW 7 =80x25 monitor */
/* */
/* setcurs(col, row) sets the cursor position */
/* indicated by 'col' and 'row' */
/* */
/* rcurspos() returns the current cursor position */
/* */
/* rcharattr() returns the character and attribute at */
/* the current cursor location */
/* */
/* rchar() returns the character at the current */
/* cursor position */
/* */
/* rattr() returns the attribute at the current */
/* cursor position */
/* */
/* wcharattr(c, color) writes character and its */
/* attribute to the current cursor loc */
/* */
/* wcharonly(c) writes character to the current */
/* cursor position */
/* */
/* wdot(x,y,color) writes a dot specified by x,y in */
/* color */
/* */
/* rdot(x,y) returns color of dot located at x,y */
/* */
/* setborder(color) sets the border color */
/* */
/* BLACK 0 RED 4 DARK_GREY 8 LIGHT_RED 12 */
/* BLUE 1 MAGENTA 5 LIGHT_BLUE 9 LIGHT_MAGENTA 13 */
/* GREEN 2 BROWN 6 LIGHT_GREEN 10 YELLOW 14 */
/* CYAN 3 LIGHT_GREY 7 LIGHT_CYAN 11 WHITE 15 */
/* */

```

```

/* setpalette( palette) sets palette color in medium */
/* resolution color mode */
/* */
/* medcolor( bckgnd, border ) sets the background and */
/* border colors in medres mode */
/* Only works on active page, and */
/* sets entire screen */
/* */
/* selectpage(page) selects active display page */
/* */
/* wstr( message, color ) */
/* writes the characters or text string pointed to by */
/* the pointer message, using the foreground color */
/* specified. ( Actually, the background color can */
/* also be specified. The various modes are, */
/* */
/* bits 0 - 2 specify the foreground color */
/* bit 3 specifies the intensity, 0 = normal */
/* bits 4 - 6 specify the background color */
/* bit 7 specifies blinking, 0 = non-blinking*/
/* */
/* The cursor is moved after each character is written.*/
/* The text should not contain any control characters, */
/* eg, don't use /n */
/* Typical use is, */
/* */
/* static char *text = "Have a nice morning."; */
/* */
/* wstr( text, 4 ); */
/* puts("/n"); */
/* */
/* */
/* scrollup(tlr,tlc,brr,brc,attr,lines) scrolls up */
/* active display area given by topleftrow,*/
/* topleftcolumn, bottomrightrow, */
/* bottomrightcolumn, using attr as a */
/* color for the bottom line, scrolling */
/* the number of lines (0=all) */
/* */
/* scrolldown(tlr,tlc,brr,brc,attr,lines) scrolls down */
/* the active display area. */
/* */
/* clear_window(tlr,tlc,brr,brc,attr) clears the */
/* display window area */
/* */
/* line(x1,y1,x2,y2,lcolor) draws a line between */
/* co-ordinates in the color specified */
/* */
/* circle ( xcentre, ycentre, radius, color ) draws */
/* a circle in the specified color. Works */
/* only in screen mode 4, 320*200Color */
/* */
/* NOTES ON THE USE OF VIDEO.LIB */
/* This has been implemented as a library. All the */
/* functions listed here can be called from your C */
/* programs. To do this however, the following guide */
/* outlines should be adhered to! */
/* */
/* 1: Incorporate the following declarations at the */
/* start of your C program. */
/* */
/* extern union REGS regs; */
/* extern unsigned char activepage; */
/* extern unsigned char columns; */

```

```

/* extern unsigned char screenmode; */
/* */
/* 2: At linking time, specify the inclusion of */
/* CHHELP.LIB */
#include <stdio.h> /* for the screen rout's */
#include <conio.h> /* used for outp() */
#include <dos.h> /* used for int86() */
#include <math.h> /* for circle routine */
union REGS regs; /* programming model 8088 */
unsigned char activepage; /* current video screen */
unsigned char columns=79; /* number of columns */
unsigned char screenmode=2; /* display mode, 80x25 col */
/* getmode() is a function that finds out the current */
/* display page number (any one of eight), the screen */
/* mode currently in use, and the number of columns */
/* (40, 80 etc) */

getmode() {
    regs.h.ah = 15; int86( 0x10, &regs, &regs);
    activepage = regs.h.bh; screenmode = regs.h.al;
    columns = regs.h.ah;
}

/* setmode() is a function that sets the display mode */
/* of the video display. First change the value of the */
/* global variable 'screenmode', then call the function */
/* 'setmode()'. It clears the display page. */
setmode() {
    regs.h.ah=0; regs.h.al=screenmode & 7;
    int86(0x10, &regs, &regs);
}

setcurs(col, row)
unsigned int col, row;
{
    getmode(); regs.h.ah = 2; regs.h.dh = row;
    regs.h.dl = col; regs.h.bh = activepage;
    int86( 0x10, &regs, &regs);
}

rcurspos() {
    getmode(); regs.h.ah = 3; regs.h.bh = activepage;
    int86( 0x10, &regs, &regs); return( regs.x.dx );
    /* row=regs.h.dh, column=regs.h.dl */
}

rcharattr() {
    getmode(); regs.h.ah = 8;
    int86(0x10, &regs, &regs); return( regs.x.ax );
    /* attribute=regs.h.ah, character=regs.h.al */
}

rchar() {
    getmode(); regs.h.ah = 8;
    int86(0x10, &regs, &regs); return( regs.h.al );
}

rattr() {
    getmode(); regs.h.ah = 8;
    int86(0x10, &regs, &regs); return( regs.h.ah );
}

wcharattr(c, color)
char c;

```

```

unsigned int color;
{
    getmode(); regs.h.ah = 9;
    regs.h.bh = activepage; regs.x.cx = 1;
    regs.h.al = c; regs.h.bl = color;
    int86( 0x10, &regs, &regs);
}

wcharonly(c)
char c;
{
    getmode(); regs.h.ah = 10;
    regs.h.bh = activepage; regs.x.cx = 1;
    regs.h.al = c; int86(0x10, &regs, &regs);
}

wdot( x, y, color )
unsigned int x, y, color;
{
    getmode();
    switch( screenmode ) {
        case 4:
        case 5:
        case 6:
            regs.h.ah = 12; regs.h.bh = 0;
            regs.x.dx = y; regs.x.cx = x;
            regs.h.al = color; int86( 0x10, &regs, &regs);
            break;
        default:
            break;
    }
}

rdot( x, y)
unsigned int x, y;
{
    getmode();
    switch( screenmode ) {
        case 4:
        case 5:
        case 6:
            regs.h.ah = 13; regs.h.bh = 0;
            regs.x.dx = y; regs.x.cx = x;
            int86(0x10, &regs, &regs);
            return ( regs.h.al );
            break;
        default:
            return ( -1 );
            break;
    }
}

setborder(color)
unsigned int color;
{
    outp( 0x3d9, color & 0x0f );
}

setpalette( palette )
int palette;
{
    getmode();
    if( screenmode <> 4 )
        return( -1 );
}

```

```

    regs.h.ah = 0x0b; regs.h.bh = 1; regs.h.bl = palette & 1;
    int86( 0x10, &regs, &regs );
}

medcolor( bckgnd, border )
int bckgnd, border;
{
    getmode();
    if( screenmode <> 4 )
        return( -1 );
    regs.h.ah = 0x0b; regs.h.bh = 0;
    regs.h.bl = (bckgnd << 4) + border;
    int86( 0x10, &regs, &regs );
}

selectpage(page)
unsigned int page;
{
    getmode();
    switch( screenmode ) {
        case 0:
        case 1:
            page = page & 7;
            break;

        case 2:
        case 3:
        case 7:
            page = page & 3;
            break;

        default:
            page = 0;
            break;
    }
    regs.h.ah = 5; regs.h.al = page;
    int86(0x10, &regs, &regs);
}

wstr( message, color )
char *message;
unsigned char color;
{
    unsigned int rowpos, colpos;
    getmode(); rcurspos();
    colpos = regs.h.dl; rowpos = regs.h.dh;
    if ( screenmode != 1 && screenmode != 3 )
        return ( -1 );
    while ( *message ) {
        wcharattr( *message, color );
        ++colpos;
        if(colpos > columns) /* check for edge of screen */
        {
            colpos = 0; /* set to beginning of line */
            ++rowpos; /* increment row count */
            if( rowpos > 24 ) /* do we need to scroll? */
            {
                rowpos = 24;
                regs.h.ah = 6; /* scroll up function call */
                regs.h.al = 1; /* scroll entire screen */
                regs.h.ch = 0; /* upper left corner */
                regs.h.cl = 0; regs.h.dl = columns;
                regs.h.dh = 24; regs.h.bh = color;
                int86(0x10,&regs,&regs); /* scroll screen */
            }
        }
    }
}

```

```

        setcurs(colpos, rowpos); /* update cursor */
        ++message; /* next character in string */
    }
}

scrollup( tlr, tlc, brr, brc, attr, lines )
unsigned int tlr, tlc, brr, brc, attr, lines;
{
    union REGS regs;
    regs.h.ah = 6; regs.h.al = lines;
    regs.h.bh = attr; regs.h.ch = tlr;
    regs.h.cl = tlc; regs.h.dh = brr;
    regs.h.dl = brc; int86( 0x10, &regs, &regs );
}

scrolldown( tlr, tlc, brr, brc, attr, lines )
unsigned int tlr, tlc, brr, brc, attr, lines;
{
    union REGS regs;
    regs.h.ah = 7; regs.h.al = lines;
    regs.h.bh = attr; regs.h.ch = tlr;
    regs.h.cl = tlc; regs.h.dh = brr;
    regs.h.dl = brc; int86( 0x10, &regs, &regs );
}

clear_window( tlr, tlc, brr, brc, attr )
unsigned int tlr, tlc, brr, brc, attr;
{
    union REGS regs;
    regs.h.ah = 6; regs.h.al = 0; regs.h.bh = attr;
    regs.h.ch = tlr; regs.h.cl = tlc; regs.h.dh = brr;
    regs.h.dl = brc; int86( 0x10, &regs, &regs );
}

line( x1, y1, x2, y2, lcolor )
int x1, y1, x2, y2, lcolor;
{
    int xx, yy, delta_x, delta_y, si, di;
    getmode();
    switch( screenmode ) {
        case 0: case 1: case 2: case 3: case 7: return(-1); break;
        default: break;
    }
    if( x1 > x2 ) {
        xx = x2; x2 = x1; x1 = xx;
        yy = y2; y2 = y1; y1 = yy;
    }
    delta_y = y2 - y1;
    if ( delta_y >= 0 )
        si = 1;
    else {
        si = -1; delta_y = -delta_y;
    }
    delta_x = x2 - x1;
    if ( delta_x >= 0 )
        di = 1;
    else {
        di = 0; delta_x = -delta_x;
    }
    if( (delta_x - delta_y) < 0 )
        steep( x1, y1, delta_x, delta_y, si, di, lcolor );
    else
        easy ( x1, y1, delta_x, delta_y, si, di, lcolor);
    return ( 0 );
}

```

```

}

steep( x1, y1, delta_x, delta_y, si, di, color )
int x1, y1, delta_x, delta_y, si, di, color;
{
    int half_delta_y, cx, dx, bx, ax, count;
    half_delta_y = delta_y / 2;
    cx = x1; dx = y1; bx = 0; count = delta_y;
newdot2: wdot( cx, dx, color );
    dx = dx + si; bx = bx + delta_x;
    if ( bx - half_delta_y <= 0 )
        goto dcount2;
    bx = bx - delta_y; cx = cx + di;
dcount2: --count;
    if ( count >= 0 )
        goto newdot2;
}

easy( x1, y1, delta_x, delta_y, si, di, color )
int x1, y1, delta_x, delta_y, si, di, color;
{
    int half_delta_x, cx, dx, bx, ax, count;
    half_delta_x = delta_x / 2;
    cx = x1; dx = y1; bx = 0; count = delta_x;
newdot:
    wdot( cx, dx, color );
    cx = cx + di; bx = bx + delta_y;
    if ( bx - half_delta_x <= 0 )
        goto dcount;
    bx = bx - delta_x; dx = dx + si;
dcount:
    --count;
    if ( count >= 0 )
        goto newdot;
}

circle ( xcentre, ycentre, radius, color )
int xcentre, ycentre, radius, color;
{
    int xfirst, yfirst, xsecond, ysecond, totalpoints = 16;
    float angle, DA;
    getmode();
    if ( screenmode != 4 )
        return (-1 );
    DA = 6.28318 / totalpoints;
    xfirst = xcentre + radius;
    yfirst = ycentre;
    for( angle = DA; angle <= 6.28318; angle = angle + DA ) {
        xsecond = xcentre + radius * cos(angle);
        ysecond = ycentre + radius * sin(angle) * .919999;
        line( xfirst, yfirst, xsecond, ysecond, color );
        xfirst = xsecond;
        yfirst = ysecond;
    }
    line( xfirst, yfirst, xcentre+radius, ycentre, color );
    return ( 1 );
}

```

Copyright Brian Brown, 1986-1999. All rights reserved.

[INDEX](#)
[◀ Previous](#)